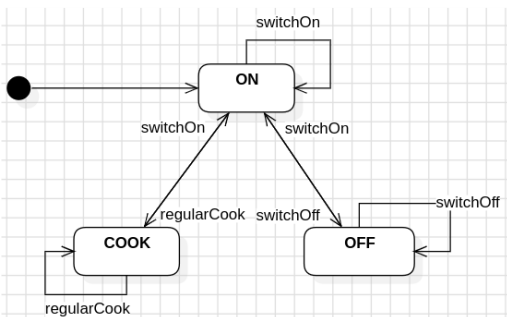


Partie I - Analyse du Robot de Cuisine

Question 1

CookBot
+ON: int {readOnly} +OFF: int {readOnly} +COOK: int {readOnly} +SLOW_COOK {readOnly} -state: int
+switchOn(): void +switchOff(): void +regularCook(): void

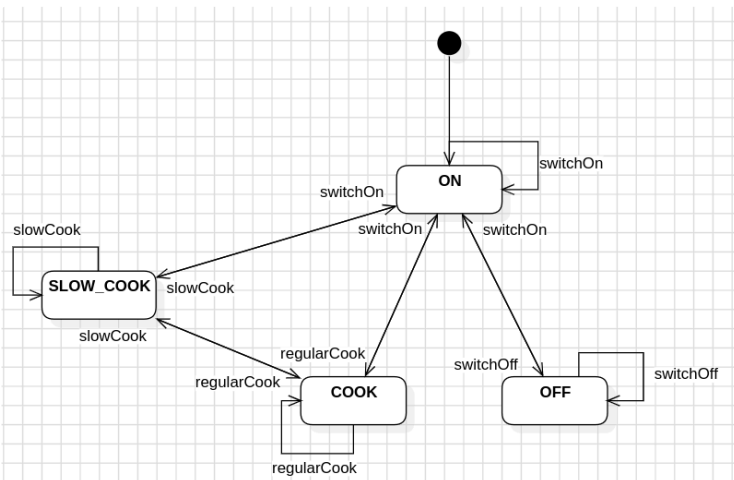
Question 2



Partie II - Et si on ajoutait une nouvelle fonctionnalité?

Question 3

CookBot
+ON: int {readOnly} +OFF: int {readOnly} +COOK: int {readOnly} +SLOW_COOK {readOnly} -state: int
+switchOn(): void +switchOff(): void +regularCook(): void



Question 5

Du côté de la modélisation, le robot ne peut pas être éteint lorsqu'il est dans le mode `SLOW_COOK` et `COOK` ce qui pose un problème si l'utilisateur a lancé le programme `SLOW_COOK` ou `COOK` par mégarde et qu'il ne peut pas l'arrêter.

Du côté de l'implémentation, tous les cas ne sont pas traités dans les méthodes.

Par exemple dans la méthode : `switchOff()` , si le robot est dans l'état `SLOW_COOK` , l'utilisateur n'est pas prévenu que le robot est toujours dans l'état `SLOW_COOK` et qu'il ne peut pas s'éteindre, ce qui peut-être dangereux, si l'utilisateur ne se rend pas compte que le robot n'est pas éteint.

De plus il manque aussi le traitement des cas non existants (else).

Il existe aussi des cas redondants.

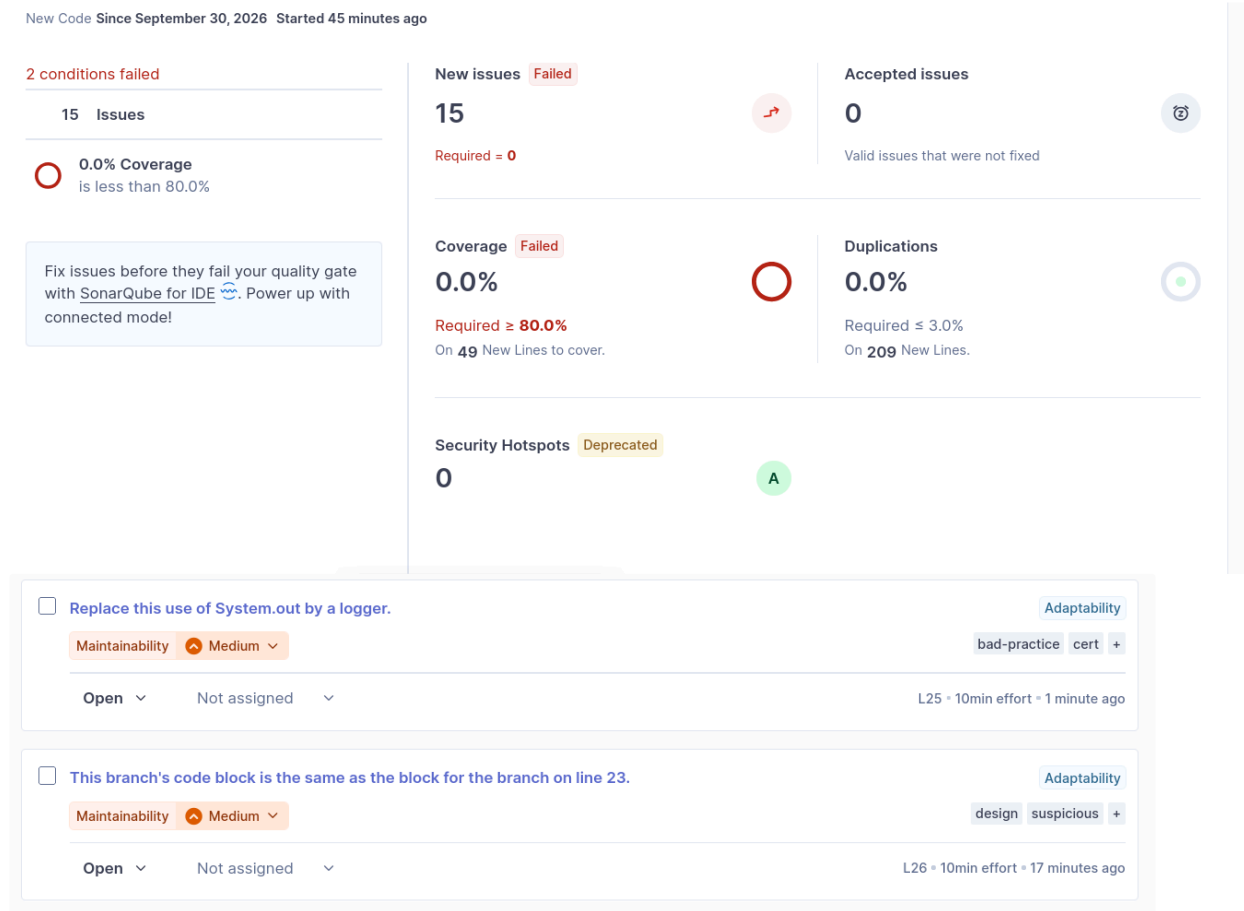
Partie III - Analyse de la qualité du code

Question 6

Il y a en tout 15 problèmes critiques résumés en deux types :

- Remplacer print classique par un Logger
- Il y a plusieurs lignes de code redondantes

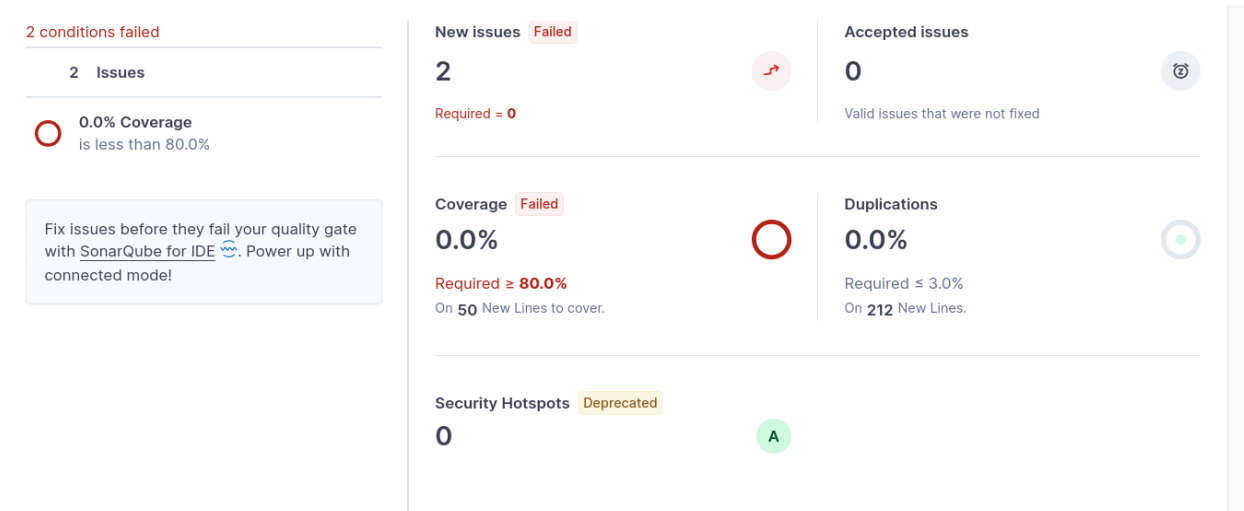
On peut donc en déduire qu'une seule partie de l'analyse a été prévue.



Partie IV - Mise en place d'un logger

Question 7

La qualité du code s'améliore à la place d'obtenir 15 problèmes on obtiens uniquement 2 problèmes.



Partie V - Test et couverture du code

Question 8

Il y a le même nombre de problèmes mais le pourcentage de couverture est plus élevé.

Partie VI - Refactoring du code

Question 9

Le principal intérêt du state pattern est de basculer d'un état à un autre en prenant en compte l'état précédent sans avoir besoin de savoir par quel état intermédiaire passer.

Dans notre cas cela pourrait être utile de passer de l'état `COOK` à l'état `OFF` sans avoir besoin de savoir qu'il faut transiter par l'état `ON`.

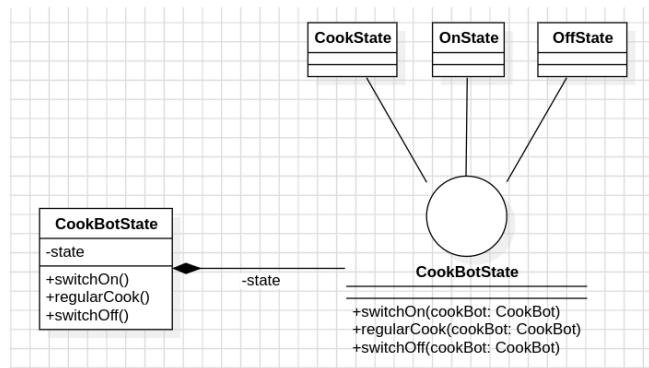
Question 10

Le diagramme de classes donné dans la Figure 2 semble respecter le modèle général donné dans le patron de conception :

- Context : `CookBot`
- State : `CookBotState`
- Concrete States : `CookState`, `OnState`, `OffState`

Utiliser une interface plutôt qu'une classe abstraite semble plus approprié dans cette situation car les états n'ont pas beaucoup de comportement en communs donc chaque état est autonome.

Question 11



Question 12

Au final le code passe le test de qualité sur SonarQube. Les seuls warnings sont ainsi des lignes de code répétées.

Quality gate

Passed

The last analysis has warnings. [See details](#)

New CodeOverall Code

Security0 Open issuesA

Reliability0 Open issuesA

Maintainability3 Open issuesA

Accepted issues0Valid Issues that were not fixed

Coverage0.0%On 58 lines to cover.

Duplications0.0%On 409 lines.

Security Hotspots Deprecated0A

src/main/java/fr/esisar/cookbot/CookBot.java

☐ This branch's code block is the same as the block for the branch on line 26.

MaintainabilityMedium

designsuspicious

OpenNot assigned

L29 • 10min effort • 1 day ago

☐ This branch's code block is the same as the block for the branch on line 26.

MaintainabilityMedium

designsuspicious

OpenNot assigned

L33 • 10min effort • 1 day ago

☐ This branch's code block is the same as the block for the branch on line 56.

MaintainabilityMedium

designsuspicious

OpenNot assigned

L59 • 10min effort • 1 day ago

3 of 3 shown